m04l1

November 9, 2026

0.0.1 Title: Ecommerce Orders.

This notebook is for cleaning up the list of orders. Only alter df_clean, and run every cell in order. The rows have already been dropped from 500 to 466 due to missing unit_prices and impossible shipping_costs.

[20]:
```
import pandas as pd
df = pd.read_csv('m4_ecommerce_orders.csv')

display(df.shape)
display(df.dtypes)
display(df.isna().sum())
display(df.describe())
display(df.head())
```

```
(500, 8)
```

```
order_id              str
order_date            str
category              str
region                str
quantity            int64
unit_price        float64
shipping_cost     float64
payment_method        str
dtype: object
```

```
order_id          0
order_date        0
category          0
region            0
quantity          0
unit_price       21
shipping_cost     0
payment_method    0
dtype: int64
```

```
         quantity  unit_price  shipping_cost
count  500.000000  479.000000     500.000000
mean     5.088000  189.523591      12.179560
std      2.653741  115.619862       8.305565
```

```
min       1.000000    5.070000     -22.560000
25%       3.000000   87.805000       5.515000
50%       5.000000  177.790000      12.620000
75%       7.000000  293.770000      19.017500
max       9.000000  398.810000      24.980000
```

```
   order_id  order_date     category   region  quantity  unit_price  \
0  ORD00001   4/12/2026         Home     East         3      377.52
1  ORD00002   3/15/2026         Toys     East         8       42.36
2  ORD00003  10/21/2026      Apparel     East         1       85.39
3  ORD00004   3/15/2026  Electronics     East         7       64.75
4  ORD00005    3/8/2026         Toys  Central         3      147.53

   shipping_cost payment_method
0           9.38         paypal
1           1.57           card
2           1.64           card
3           3.44           card
4           7.71           card
```

[30]:
```
# the missing ones, in one line
print(df.isna().sum())

# the impossible ones, one check each. Two are done for you.
print((df['shipping_cost'] < 0).sum(), 'negative shipping costs')
print((df['quantity'] < 1).sum(), 'quantities below one')

# write one more check of your own for another column
print((df['unit_price'] < 0).sum(), 'negative unit prices')
```

```
order_id          0
order_date        0
category          0
region            0
quantity          0
unit_price       21
shipping_cost     0
payment_method    0
dtype: int64
13 negative shipping costs
0 quantities below one
0 negative unit prices
```

[44]:
```
# run this before you decide which fix to use
before = df['quantity'].mean()
after  = df.dropna(subset=['unit_price'])['quantity'].mean() # means to drop N/
  ↪A rows in that column unit_price
print(before, after)
```

```
# then apply your two fixes and save the result as df_clean

# WHAT: 21 blank unit prices
# DID: Dropped those 21 rows
# COSTS: Lost actual customer details for that order, like payment, date, category, etc.
df_clean = df.dropna(subset=['unit_price'])

# WHAT: 13  Negative shipping costs
# DID: Got rid of the rows with negative shipping costs
# COSTS: This also lost 13 real order details
df_clean = df_clean[df_clean['shipping_cost'] >= 0] # gets rid of rows with negative values

print(df_clean.shape)
display(df_clean.head())
print(before, df_clean['quantity'].mean())
```

```
5.088 5.079331941544885
(466, 8)
```

	order_id	order_date	category	region	quantity	unit_price	shipping_cost	payment_method
0	ORD00001	4/12/2026	Home	East	3	377.52	9.38	paypal
1	ORD00002	3/15/2026	Toys	East	8	42.36	1.57	card
2	ORD00003	10/21/2026	Apparel	East	1	85.39	1.64	card
3	ORD00004	3/15/2026	Electronics	East	7	64.75	3.44	card
4	ORD00005	3/8/2026	Toys	Central	3	147.53	7.71	card

```
5.088 5.070815450643777
```

[42]:
```
# the pattern: group by one column, average another
print(df_clean.groupby('category')['quantity'].mean().sort_values(ascending=False))

print()

# now do the same for region and shipping_cost
print(df_clean.groupby('region')['shipping_cost'].mean().sort_values(ascending=False))
```

```
category
```

```
Toys           5.312500
Electronics    5.213483
Home           5.183673
Apparel        4.966667
Grocery        4.666667
Name: quantity, dtype: float64

region
South      13.307869
West       13.027647
Central    12.885648
East       12.266667
Name: shipping_cost, dtype: float64
```